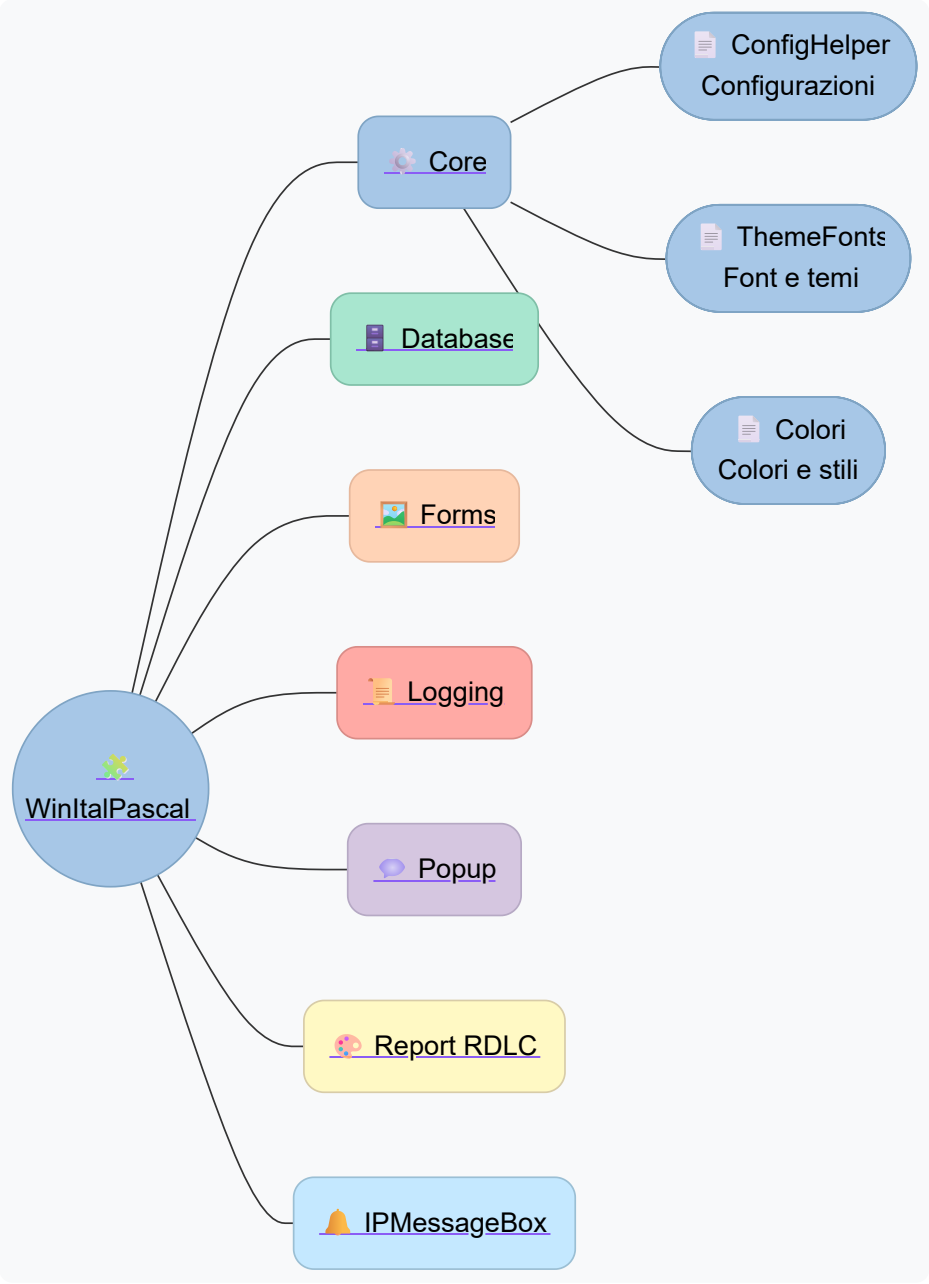
Ital Pascal


ConfigHelper
Configurazioni
Core
ThemeFonts
Font e temi
Database
Colori
Colori e stili
Forms
Logging
WinItalPascal
Popup
Report RDLC
IPMessageBox

WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

💠 Modulo `ColoriDgv` – Spiegazione completa

1. Scopo generale del codice

Il modulo `ColoriDgv` fornisce una serie di helper per colorare in modo dinamico le colonne di una o più `DataGridView` in un'applicazione WinForms.

- Si può scegliere un tema predefinito (pastello, soft-blu, soft-verde, Office 2016 "Of24").
- Si può specificare un set di colori personalizzato per ogni colonna.
- È possibile colorare solo colonne selezionate o "saltare" alcune colonne.
- Viene fornito un metodo per ripristinare i colori di default.

In pratica, il modulo è un toolkit di styling per le griglie di dati, utile per rendere l'interfaccia più leggibile e coerente con il tema dell'applicazione.

---

2. Componenti principali e funzioni chiave

Sezione	Funzione	Descrizione
Enum	`ColSpec`, `TemaM`, `TemaProf`	Enumerazioni per indicare "salta" (colonna non colorata), temi di colore generico e temi professionali.
`ColoraDgv`	`Public Sub ColoraDgv(ParamArray griglie() As DataGridView)`	Colora le colonne di ogni griglia usando i colori predefiniti di `GridUtility.ColoraColonne`.
`ColoraDgvColorato`	`Public Sub ColoraDgvColorato(colori As (Colori.ColoreTipo, Colori.ColoreTipo,`	Stessa cosa di `ColoraDgv` ma con colori passati come tuple.

Sezione	Funzione	Descrizione
	`Colori.ColoreTipo), ParamArray griglie() As DataGridView)`	
`ColoraColonneSpec`	`Public Sub ColoraColonneSpec(dgv As DataGridView, colonne() As Integer, Optional coloreSfondo As Color = Nothing, Optional coloreTesto As Color = Nothing)`	Colora solo le colonne specificate tramite un handler `CellFormatting`.
`ColoraDgvMod`	`Public Sub ColoraDgvMod(dgv As DataGridView, tema As TemaM)`	Applica un tema generico (Pastello, SoftBlu, SoftVerde) a tutte le colonne.
`ColoraDgvModMulti`	`Public Sub ColoraDgvModMulti(tema As TemaM, ParamArray griglie() As DataGridView)`	Varianti “multi” per più griglie.
`ColoraDgvOf24`	`Public Sub ColoraDgvOf24(dgv As DataGridView, tema As TemaProf)`	Applica il tema Office 2016 “Of24” (Office Blue, Gray, Accent).
`ColoraDgvOf24Multi`	`Public Sub ColoraDgvOf24Multi(tema As TemaProf, ParamArray griglie() As DataGridView)`	Varianti “multi” per più griglie.
`ResetColoriDgv`	`Public Sub ResetColoriDgv(ParamArray griglie() As DataGridView)`	Ripristina colori di default (bianco sfondo, nero testo) e rimuove eventuali handler.
`ColoraDgvCustom`	`Public Sub ColoraDgvCustom(dgv As DataGridView, ParamArray colori() As Object)`	Colora colonne in base a un array di colori o a `ColSpec.Salta`.
`ColoraDgvCustomM`	`Public Sub ColoraDgvCustomM(colori() As Object, ParamArray griglie() As DataGridView)`	Varianti “multi” per più griglie.

Dettagli di implementazione

- **ParamArray**: consente di passare un numero variabile di `DataGridView` o colori.
- **Tuple**: `ColoraDgvColorato` accetta una tupla di tre colori, rendendo l’API più leggibile.
- **Eventi**: `ColoraColonneSpec` aggiunge un handler `CellFormatting` per colorare dinamicamente le celle in base all’indice della colonna.
- **Colori personalizzati**: `ColoraDgvCustom` gestisce sia oggetti `Color` sia l’enum `ColSpec` per “saltare” una colonna.

- **Reset**: `ResetColoriDgv` pulisce sia le colonne che le celle e tenta di rimuovere gli handler (anche se `RemoveHandler dgv.CellFormatting, Nothing` non è corretto; bisogna tenere un riferimento all'handler).

---

3. Flusso di esecuzione tipico

1. **Scelta del metodo**
   L'applicazione chiama uno dei metodi pubblici in base al requisito:
   - Tema generico → `ColoraDgvMod` o `ColoraDgvModMulti`.
   - Tema Office → `ColoraDgvOf24` o `ColoraDgvOf24Multi`.
   - Colori personalizzati → `ColoraDgvCustom` o `ColoraDgvCustomM`.
   - Colonne specifiche → `ColoraColonneSpec`.
   - Reset → `ResetColoriDgv`.
2. **Verifica di input**
   Ogni metodo controlla che la griglia non sia `Nothing` e che abbia colonne. Se la condizione non è soddisfatta, il metodo esce (`Exit Sub` o `Continue For`).
3. **Calcolo dei colori**
   - Per i temi, viene usato un `Select Case` per impostare tre colori (`c1`, `c2`, `c3`).
   - Per i colori personalizzati, si itera sull'array di colori e si applicano i valori corrispondenti.
4. **Applicazione dei colori**
   - Per ogni colonna (`For i = 0 To dgv.Columns.Count - 1`), si imposta `DefaultCellStyle.BackColor` e `ForeColor`.
   - Nel caso di `ColoraColonneSpec`, si aggiunge un handler `CellFormatting` che cambia lo stile solo per le colonne indicate.
5. **Reset**
   `ResetColoriDgv` ripristina i colori di default e pulisce gli handler.

---

4. Esempi di utilizzo

```
' 1. Tema generico
ColoraDgvMod(FatturaDataGrid, TemaM.Pastello)

' 2. Tema Office
ColoraDgvOf24(OrdiniDataGrid, TemaProf.Of24)

' 3. Colori personalizzati con "salta"
ColoraDgvCustom(OrdiniDataGrid, Color.LightBlue, ColSpec.Salta,
Color.LightGreen)

```

```
' 4. Colonne specifiche
ColoraColonneSpec(ClientiDataGrid, {1, 3}, Color.Aquamarine,
Color.DarkBlue)

' 5. Reset
ResetColoriDgv(OrdiniDataGrid, ClientiDataGrid, FatturaDataGrid)
```

4.1 Colore personalizzato (`ColoraDgvCustom` / `ColoraDgvCustomM`)

1. **Input**: una `DataGridView` e un array di oggetti (`Color` o `ColSpec.Salta`).
2. Per ogni colonna:

- Se l'oggetto è `ColSpec.Salta` , la colonna resta bianca/nera.
- Se è un `Color` , la colonna assume quel colore di sfondo.
- Se non c'è valore, la colonna viene ripristinata a bianco/nero.

1. `ColoraDgvCustomM` semplicemente chiama `ColoraDgvCustom` per ciascuna griglia fornita.

> ### 🔹 Conclusione
>
> Il modulo `ModColoriDgv` è un toolkit versatile per la personalizzazione visiva delle `DataGridView` . Offre sia soluzioni predefinite (temi) sia la possibilità di definire colori su misura, con un'attenzione particolare alla gestione di colonne "saltate" e alla possibilità di ripristinare lo stato originale. Effettuato correzione (gestione degli handler, definizione di `ColoreTipo`) il codice diventa robusto e pronto all'uso in progetti WinForms.



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

Analisi del codice `ConfigHelper`

1. Scopo generale

Il **classe** `ConfigHelper` è un helper statico (non ereditabile) che fornisce un metodo di utilità per recuperare una stringa di connessione dal file di configurazione dell'applicazione (`App.config` o `Web.config`).
In pratica, consente di ottenere in modo centralizzato e sicuro la stringa di connessione necessaria per connettersi a un database, evitando di ripetere il codice di accesso alla configurazione in più punti dell'applicazione.

---

2. Componenti chiave

Elemento
`Public NotInheritable Class ConfigHelper` Classe pubblica che non può essere ereditata (`NotInheritable`). Garantisce che la classe rimanga un "singleton" di utilità.
`Private Sub New()` Costruttore privato. Impedisce l'istanza della classe; tutti i membri devono essere statici (`Shared`).
`Public Shared Function GetConnectionString(Optional name As String = "MiaConnessione") As String` Metodo pubblico e statico che restituisce la stringa di connessione. Accetta un parametro opzionale `name` con valore di default `"MiaConnessione"` .
`Dim cs = System.Configuration.ConfigurationManager.ConnectionStrings(name)` Recupera l'oggetto `ConnectionStringSettings` dal file di configurazione. Se la chiave non esiste, `cs` sarà `Nothing` .
`If cs Is Nothing Then ... Throw New Exception(...)` Controllo di errore: se la stringa di connessione non è presente, viene lanciata

Elemento
un’eccezione. L’eccezione è generica (`Exception`); si potrebbe usare un tipo più specifico (`ConfigurationErrorsException`).
`Return cs.ConnectionString` Restituisce la proprietà `ConnectionString` dell’oggetto recuperato.Il valore è una stringa che può essere usata direttamente per creare una connessione al database.

2.1 Componenti principali

Elemento	Descrizione
`Imports System.Configuration`	Permette di accedere alla classe `ConfigurationManager` .
`Imports System.Windows.Forms`	Per mostrare finestre di dialogo (`IPMessageBox`).
`Public NotInheritable Class ConfigHelper`	Classe finale che raccoglie i metodi di utilità.
`Private Sub New()`	Costruttore privato: impedisce la creazione di istanze.

3 Funzione `GetConnectionString`

Parametro

- `name` (opzionale) – Nome della chiave da cercare.
  Valore di default: `"MiaConnessione"` .

Logica

1. **Recupero della stringa**

   ```
   1  Dim cs = ConfigurationManager.ConnectionStrings(name)
   ```

   Se la chiave non esiste, `cs` è `Nothing` .
2. **Validazione**

   ```
   1  If cs Is Nothing OrElse
      String.IsNullOrWhiteSpace(cs.ConnectionString) Then
   ```

   Se la chiave è assente o la stringa è vuota, viene:
   - Costruito un messaggio di errore dettagliato.
   - Mostrato tramite `IPMessageBox.Show` .
   - Lanciata una `ConfigurationErrorsException` con lo stesso messaggio.

3. **Successo**
   Se la chiave è presente e non vuota, la funzione restituisce `cs.ConnectionString`.
4. **Gestione eccezioni**
   - `Catch ex As ConfigurationErrorsException` → rilancia l'eccezione (non viene modificata).
   - `Catch ex As Exception` → mostra un messaggio generico di errore e rilancia l'eccezione.

3.1 Flusso di esecuzione

```
GetConnectionString()
    ├─ Recupera ConnectionStringSettings
    │   └─ Se non trovato → errore
    ├─ Se stringa vuota → errore
    ├─ Altrimenti → restituisce la stringa
    └─ Gestione eccezioni
```

4. Flusso di esecuzione

1. **Chiamata del metodo**
   Quando un altro componente chiama `ConfigHelper.GetConnectionString()`, il metodo viene eseguito senza creare un'istanza della classe (grazie al costruttore privato).
2. **Recupero della configurazione**
   Il metodo richiama `ConfigurationManager.ConnectionStrings(name)` per cercare la stringa di connessione con la chiave specificata (`name`).
3. **Verifica dell'esistenza**
   Se l'oggetto restituito è `Nothing`, il metodo solleva un'eccezione con un messaggio che indica che la chiave non è stata trovata nel file di configurazione.
4. **Restituzione del valore**
   Se la stringa è presente, il metodo restituisce la proprietà `ConnectionString`, che è la stringa effettiva da utilizzare per la connessione al database.

> **Conclusione**
>
> Il codice fornisce un modo semplice e centralizzato per ottenere le stringhe di connessione dal file di configurazione. È strutturato come una classe di utilità non ereditabile, con un costruttore privato e un metodo statico. Il flusso è lineare: recupera la configurazione, verifica la sua esistenza e restituisce la stringa.
> `ConfigHelper` è un helper semplice ma robusto per accedere alle stringhe di

connessione.
Garantisce che l’applicazione non proceda con una configurazione incompleta, fornendo messaggi chiari all’utente e lanciando eccezioni appropriate per la gestione degli errori.



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

Analisi del codice VB.NET

1. Scopo generale

Il file definisce una **classe di utilità** chiamata `ThemeFonts`.
La sua funzione principale è fornire un set di oggetti `Font` predefiniti che possono essere ri-utilizzati in tutta l'applicazione per garantire un aspetto coerente (tema di font).
Essendo `NotInheritable` (sealed) e con un costruttore privato, la classe non può essere istanziata né estesa: tutti i membri sono **statici** (`Shared`) e vengono creati una sola volta all'avvio dell'applicazione.

---

2. Componenti chiave

Elemento	Tipo	Descrizione
`Public NotInheritable Class ThemeFonts`	Classe	Classe sealed, non estendibile.
`Private Sub New()`	Costruttore privato	Previene la creazione di istanze.
`Public Shared ReadOnly DefaultFont`	`Font`	Font di base, "Microsoft Sans Serif", 10pt, normale.
`Public Shared ReadOnly DefaultFontBold`	`Font`	Stesso font ma in grassetto.
`Public Shared ReadOnly TitleFont`	`Font`	Font per titoli, "Segoe UI", 12pt, grassetto.

Dettagli di implementazione

- `Font` è una classe di `System.Drawing` che rappresenta un tipo di carattere, la sua dimensione e lo stile (Regular, Bold, ecc.).
- Ogni oggetto `Font` è creato con il costruttore `New Font(String, Single, FontStyle)`.
- I campi sono `ReadOnly`: non possono essere modificati dopo l'inizializzazione, garantendo l'immutabilità del tema.
- L'uso di `Shared` rende i campi accessibili senza creare un'istanza: `ThemeFonts.DefaultFont`, `ThemeFonts.TitleFont`, ecc.

---

3. Flusso di esecuzione

1. **Caricamento dell'applicazione**
   Quando l'applicazione viene avviata, il runtime carica la classe `ThemeFonts`.
   - Il costruttore statico (generato implicitamente) crea le tre istanze di `Font` e le assegna ai campi `ReadOnly`.
2. **Accesso ai font**
   In qualsiasi punto dell'applicazione, un componente grafico può richiamare, ad esempio:

```vb
Dim lbl As New Label()
lbl.Font = ThemeFonts.TitleFont
```

   Il font viene passato per riferimento; non viene creato un nuovo oggetto ogni volta.

> ✏️ 🔹 **Conclusione**
>
> `ThemeFonts` è una semplice, ma efficace, classe di supporto per centralizzare la definizione dei font di un'applicazione VB.NET.
> Grazie al costruttore privato e ai membri `Shared ReadOnly`, garantisce un utilizzo coerente e sicuro dei font, riducendo la duplicazione di codice e facilitando la manutenzione del tema visivo.